Breaking the Black Box

Rethinking Software Testing in the Age of AI

MODULE 1

Understanding AI & Large Language Models

A Complete Beginner's Textbook

Topics Covered in This Module

- What is Artificial Intelligence?
- The Evolution of AI — From Rules to Neural Networks
- How Large Language Models Work
- Tokens, Embeddings, and Attention
- Why AI Gives Different Answers — Probability & Temperature
- Real-World Applications of AI
- Why Testing AI is Fundamentally Different

Designed for absolute beginners. No prior technical knowledge required.

Takshin Varma

AI Engineer | Author | Technical Trainer

A Note to the Learner

Welcome to Module 1. Before we dive into concepts, take a moment to understand how this textbook is designed to work for you.

This module does not assume that you have any background in computers, mathematics, or technology. Every concept introduced here will be explained from the ground up, using simple language, everyday analogies, and step-by-step reasoning.

Throughout these notes, you will encounter a variety of special sections designed to make your learning experience richer and more memorable.

Section Type	What It Means
Concept Box	A focused explanation of a new idea or term
Analogy	A real-life comparison to help you understand abstract ideas
Example	A concrete, practical illustration of a concept
Key Insight	The single most important takeaway from a section
Testing Note	How the concept relates to software quality and testing
Summary Table	A quick-reference overview at the end of a section

How to Use These Notes

Read each section slowly and without skipping. The concepts build on one another. If something is unclear, re-read the analogy section — it is designed to make the idea click. After each major section, pause and ask yourself: 'Can I explain this in my own words?' If the answer is yes, move forward. If not, re-read before continuing.

Part 1 — What Is Artificial Intelligence?

— The Foundation —

1.1 Starting With Something Familiar

To understand Artificial Intelligence, let us begin with something very ordinary — a calculator. When you type 2 + 2 into a calculator, it gives you 4. Every single time. Without exception. No matter how many times you ask, the answer is always exactly 4. The calculator is not thinking. It is not guessing. It is simply following a fixed rule that a programmer wrote long ago: add these two numbers together and display the result.

This is how most traditional software works. It follows a set of precise, rigid instructions. The instructions are written by human programmers. The computer executes them faithfully. The output is always the same for the same input. There is nothing flexible or creative about it.

Think About This

Now imagine a different kind of system. You type: "Tell me a story about a dragon who is afraid of fire."And every time you ask, you get a different story. A different dragon, a different adventure, a different ending. The system is not retrieving a pre-written story. It is creating something new each time. That is the essence of Artificial Intelligence — a computer system that goes beyond rigid rules to produce flexible, context-aware, creative outputs.

1.2 What Exactly Is Artificial Intelligence?

Artificial Intelligence, commonly abbreviated as AI, refers to computer systems that are designed to perform tasks that would normally require human-like intelligence. These tasks include understanding language, recognising patterns, making decisions, solving problems, and generating creative content.

Unlike traditional software that follows explicit instructions, AI systems learn from data. They observe thousands — or even millions — of examples and gradually build an internal model of the world. This model then allows them to respond intelligently to new situations they have never seen before.

Traditional Software	Artificial Intelligence
Follows fixed, pre-written rules	Learns patterns from data
The same input always gives the same output	Output can vary based on learning
Cannot adapt to new situations	Can generalise to new inputs
A programmer must anticipate every case	System discovers patterns automatically
Example: a calculator or login system	Example: a chatbot or image recogniser

1.3 What Can AI Actually Do?

Modern AI systems are capable of a wide and growing range of tasks. Here is a realistic overview of what AI can and cannot do:

What AI Can Do
Read and understand written text in multiple languages. Write essays, stories, code, and emails. Answer questions based on the provided information. Recognise faces in photographs. Translate between languages. Suggest what product to buy or what film to watch. Detect spam emails. Assist doctors in analysing medical images.

What AI Cannot Do (Yet)
Truly understand the world the way humans do. Feel emotions, desires, or motivations. Reason perfectly about situations it has never encountered. Guarantee that its outputs are accurate or truthful. Be held accountable in the way a human can be.

It is critically important to understand this distinction. AI systems are powerful pattern-recognition and prediction machines. They are not thinking, feeling, or understanding entities. This distinction will become very important when we discuss testing AI systems later in this module.

Part 2 — The Evolution of Artificial Intelligence

— From Simple Rules to Massive Models —

AI did not appear overnight as a complete, polished technology. It evolved gradually over decades, through four distinct phases. Each phase brought new capabilities — and new challenges for testing and quality assurance. Understanding this history is essential because many of the challenges we face in AI testing today are a direct result of how the technology evolved.

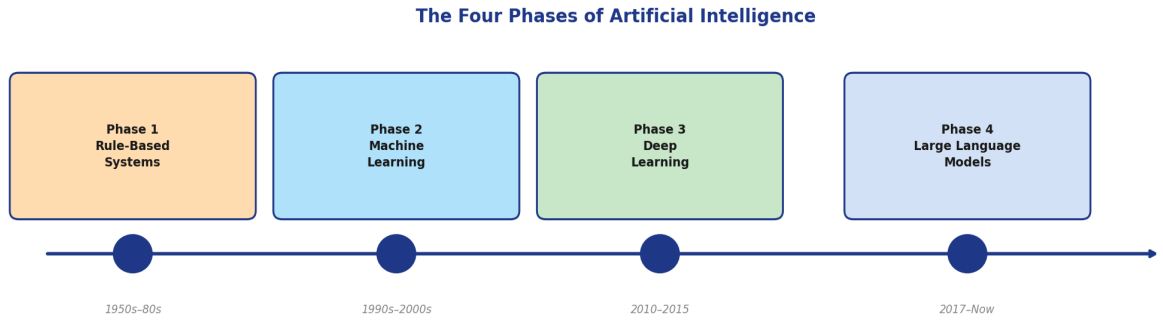

Figure 1.1 — The four phases of AI development, from rule-based systems to large language models

2.1 Phase One — Rule-Based Systems

The earliest AI systems, developed in the 1950s through the 1980s, operated entirely on rules written by human programmers. These rules were explicit, detailed, and exhaustive. The programmers had to think of every possible scenario in advance and write a rule for it.

A classic example is ELIZA, one of the earliest conversational programs created at MIT in the 1960s. ELIZA could hold a basic conversation by matching patterns in the user's input and selecting a pre-written response. If the user typed a sentence containing the word 'mother', ELIZA might respond with 'Tell me more about your family.'

Analogy — The Recipe Book

Think of a rule-based system like a very detailed recipe book. If a customer orders a chocolate cake, you look up the exact recipe and follow it step by step. If they order a lemon tart, you look up that recipe. But what if they order something that is not in the book? You have no recipe, so you cannot help. Rule-based AI had exactly this limitation — it could only handle scenarios its programmers had already anticipated.

Testing rule-based systems was relatively straightforward. Since every input led to a predictable, programmer-defined output, a tester could write clear test cases: 'If the user says X, the system should respond with Y.' Either the output matched, or it did not. Testing was deterministic.

2.2 Phase Two — Machine Learning

In the 1990s and 2000s, a fundamentally different approach emerged: machine learning. Instead of programming explicit rules, developers provided the system with large amounts of labelled data and allowed it to discover its own patterns.

The spam filter is a classic example of machine learning. Rather than a programmer writing rules like 'if the email contains the word free and mentions money, mark it as spam', the system was trained on thousands of emails labelled as either spam or not spam. Over time, it learned the statistical patterns that distinguished the two categories. When a new email arrived, the system could classify it based on the patterns it had discovered.

The Key Insight of Machine Learning

The programmer no longer needs to write the rules. The system writes its own rules by studying examples. This is a profound shift in how we build intelligent systems — and it introduces a profound shift in how we must test them.

2.3 Phase Three — Deep Learning and Neural Networks

From 2010 to 2015, advances in computing power and the availability of massive datasets enabled a new form of machine learning called deep learning. Deep learning systems use artificial neural networks — architectures loosely inspired by the structure of the human brain.

A neural network is organised into layers. The first layer receives raw input — for example, the pixels of an image. Each subsequent layer processes the information at a higher level of abstraction. By the final layer, the network has transformed the raw pixels into a meaningful output, such as 'this image contains a cat.'

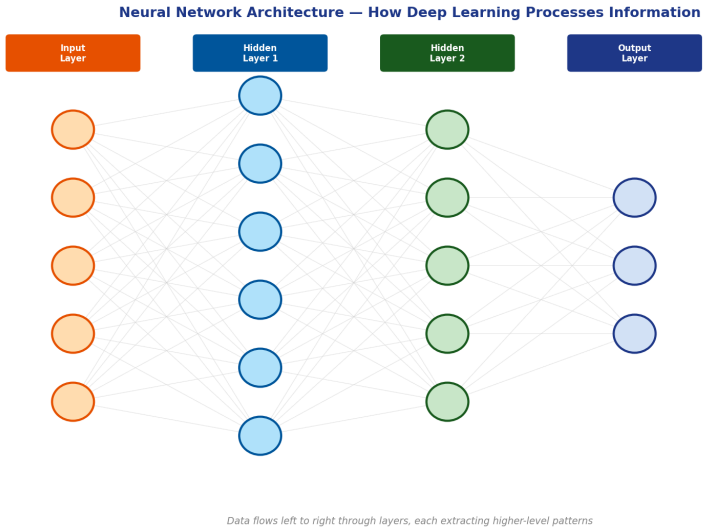

Figure 2.1 — A neural network processes data through successive layers, each extracting increasingly abstract patterns

Neural Network Layer	What It Does
Input Layer	Receives raw data (e.g., image pixels or word tokens)
Hidden Layers	Identify patterns, features, and abstract representations
Output Layer	Produces the final result (e.g., classification or generated text)

Testing Challenge — The Black Box Problem

Deep learning systems are sometimes called 'black boxes' because their internal decision-making processes are opaque. Even the engineers who build them often cannot fully explain why a particular output was produced. For testers, this means we cannot rely on logic alone. We need to test outputs empirically — running large numbers of test cases and observing patterns in the results.

2.4 Phase Four — Large Language Models

The most recent and transformative development in AI is the emergence of Large Language Models, or LLMs. These are the systems that power tools like ChatGPT, Claude, Gemini, and many others. Examples of LLMs include GPT-4, PaLM, LLaMA, and Claude.

LLMs are trained on enormous datasets — collections of books, websites, academic articles, code repositories, forum discussions, and much more. The scale of this training is almost incomprehensible. Through this training, the model develops a rich, nuanced understanding of language and the patterns of human knowledge.

The defining characteristic of an LLM is that it does not store answers. Instead, it learns the statistical relationships between words, phrases, and ideas. When you ask it a question, it generates a response by predicting what words are most likely to follow, given everything it has learned.

Key Concept — LLMs Predict, They Do Not Retrieve

This is one of the most important ideas in this entire module. When you ask an LLM a question, it does not look up the answer in a database. It generates the answer word by word, each word chosen based on statistical probability. This is why LLMs can answer questions that were never explicitly in their training data — and also why they sometimes confidently produce incorrect information.

Part 3 — How Large Language Models Work

— Inside the Machine —

Now that we understand what LLMs are and how they came to exist, let us look inside and understand how they actually function. This section explains three fundamental concepts: tokenisation, embeddings, and attention. Together, these three mechanisms explain how a language model transforms your text input into a meaningful output.

3.1 Step One — Tokenisation: Breaking Language Into Pieces

When you type a message to an AI system, the very first thing that happens is that your text is broken down into small units called tokens. Tokens are the basic building blocks that language models work with.

A token is not always a complete word. It might be an entire word, part of a word, a single punctuation mark, or even a space. The exact way text is divided into tokens depends on the tokeniser — the algorithm responsible for this process.

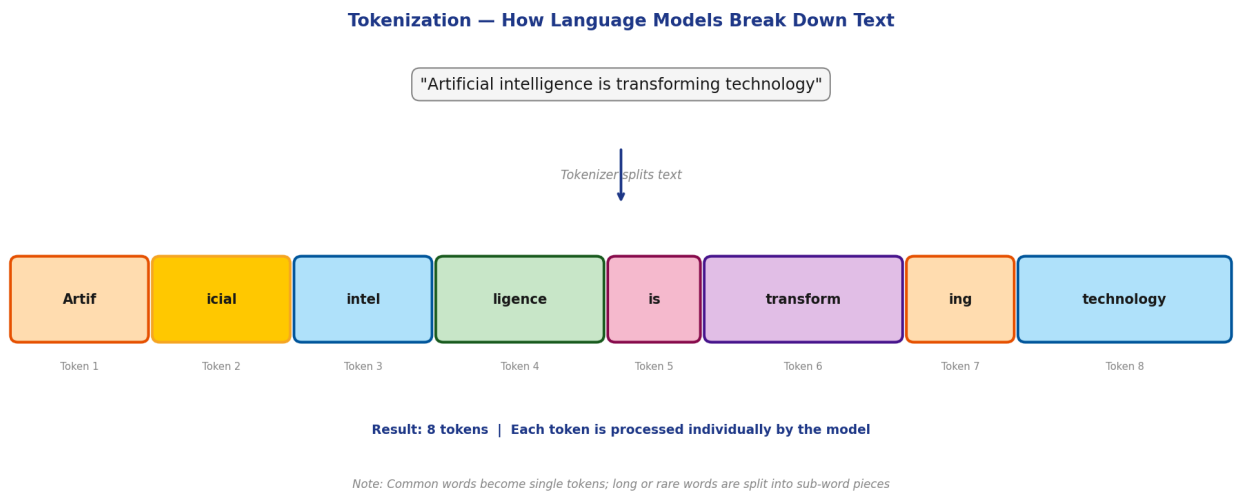

Figure 3.1 — Tokenisation splits text into discrete units that the model can process mathematically

Example — Tokenisation in Practice

Consider the sentence: "AI is powerful"A tokeniser might break this into:[AI] [is] [powerful]. Or for a longer word like "unhappiness":[un] [happi] [ness]. Each bracketed unit is a single token. The model processes each token individually and in relation to all other tokens in the input.

Why does tokenisation matter? Because the model has a maximum number of tokens it can process at once, this is called the context window. Everything outside the context window is invisible to the model. A model with a context window of 8,000 tokens, for example, cannot 'see' text that was written more than approximately 6,000 words ago in a long conversation.

For testers, this has practical implications. A system that performs well on short inputs may behave differently on very long inputs if they exceed the context window. Testing must consider the full range of input lengths.

3.2 Step Two — Embeddings: Turning Words Into Numbers

Once text has been broken into tokens, each token must be transformed into a format that the model can mathematically process. Computers do not understand words — they only understand numbers. This transformation is handled through a mechanism called embeddings.

An embedding is a list of numbers — technically called a vector — that represents a token in a mathematical space. These numbers are not arbitrary. They are carefully learned during training so that tokens with similar meanings end up with similar number values.

Analogy — A Map of Meaning

Imagine a map where every word in the English language has a location. Words with similar meanings are placed close together on the map. 'King' and 'Queen' are near each other. 'Run' and 'sprint' are near each other. 'Happiness' and 'joy' are near each other. 'Banana' and 'monarchy' are far apart. An embedding is essentially a set of coordinates on this map of meaning. By turning words into coordinates, the model can perform mathematical operations that capture semantic relationships — relationships of meaning — between concepts.

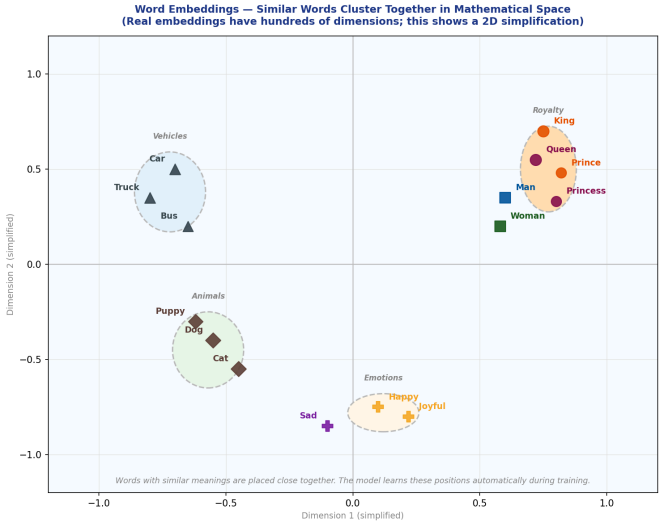

Figure 3.2 — A 2D simplification of word embeddings. Words with related meanings cluster together in the mathematical space.

One of the most famous demonstrations of the power of embeddings is the following mathematical relationship: the embedding for 'King', minus the embedding for 'Man', plus the embedding for 'Woman', produces a result very close to the embedding for 'Queen'. The model has encoded the concept of royalty and gender as mathematical relationships.

For testers, understanding embeddings helps explain why language models behave as they do with synonyms, paraphrases, and conceptually related inputs. A model may respond very similarly to 'What is the capital of France?' and 'Name the main city of France', even though the words are different, because the embeddings are similar.

3.3 Step Three — Attention: Understanding Relationships

The third and perhaps most powerful mechanism in modern language models is the attention mechanism. This is the technological breakthrough that made LLMs possible, and it was introduced in a landmark 2017 research paper titled 'Attention Is All You Need'.

To understand why attention matters, consider how earlier language models worked. They processed text sequentially — one word at a time, from left to right. By the time they reached the end of a long sentence, important information from the beginning had often been 'forgotten'.

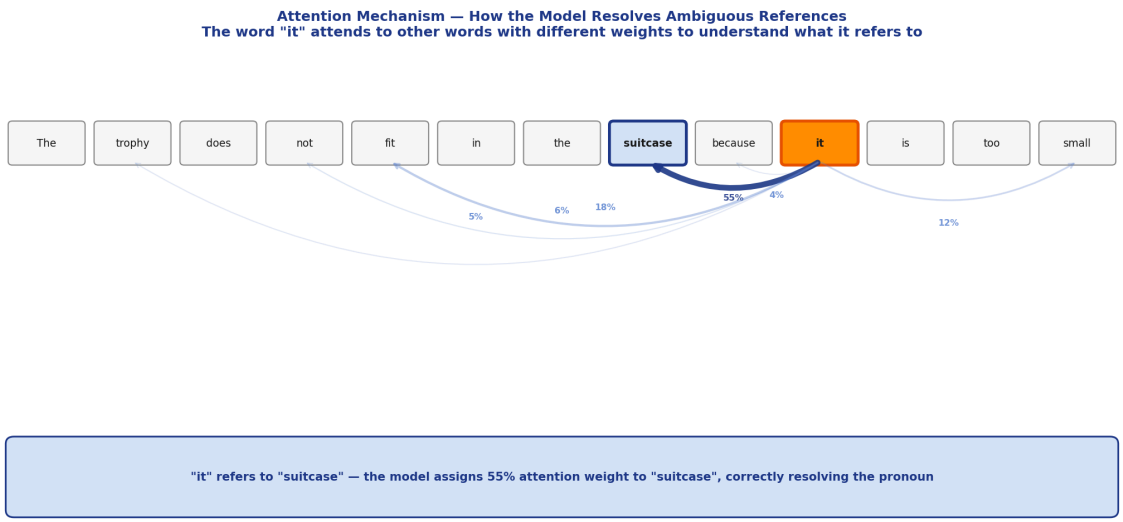

Figure 3.3 — The attention mechanism shows how much each word 'pays attention' to every other word. Here, 'it' correctly attends most strongly to 'suitcase'

A Classic Ambiguity — The Pronoun Problem

Consider this sentence: "The trophy does not fit in the suitcase because it is too small."What does the word 'it' refer to? It refers to the suitcase. But to resolve this, you need to look at the entire sentence — specifically the relationship between 'fit', 'trophy', 'suitcase', and 'small'. The attention mechanism allows the model to look at ALL words in the input simultaneously and calculate the strength of the relationship between every pair of words. When processing 'it', the model attends strongly to 'suitcase', resolving the ambiguity correctly.

This is why modern language models are so much better at understanding long, complex sentences and maintaining context across a conversation. They are constantly looking at the whole picture, not just the most recent word.

Mechanism	What It Does for the Model
Tokenization	Breaks text into manageable units that the model can process
Embeddings	Converts tokens into mathematical vectors that encode meaning
Attention	Allows the model to relate any word to any other word in context

Part 4 — Why AI Gives Different Answers

— Probability, Randomness, and Temperature —

One of the first things new users notice about AI systems is that they can give different answers to the same question on different occasions. Ask an AI to write a poem about autumn, and you will get a different poem each time. This is not a bug. It is a fundamental feature of how language models work, rooted in probability theory.

4.1 Language as Prediction

At its core, a language model is a prediction engine. Given the tokens that have appeared so far in a conversation, the model calculates the probability of every possible next token. It then selects a token based on those probabilities.

Consider this simple example. If the model has processed the sentence 'The sky is', it might calculate the following probabilities for the next word:

Candidate Next Word	Approximate Probability
blue	45%
clear	20%
bright	15%
beautiful	10%
dark	7%
other words (combined)	3%

The model does not always choose the highest probability word. If it did, every response would be the same — and quite boring. Instead, it samples from this probability distribution, which means it randomly selects a word with probability proportional to each word's score. This sampling process is repeated for every single token in the response, which might be hundreds or thousands of tokens for a long answer.

4.2 Temperature — The Randomness Control

The degree of randomness in this sampling process is controlled by a parameter called temperature. Understanding temperature is essential for anyone working with or testing AI systems.

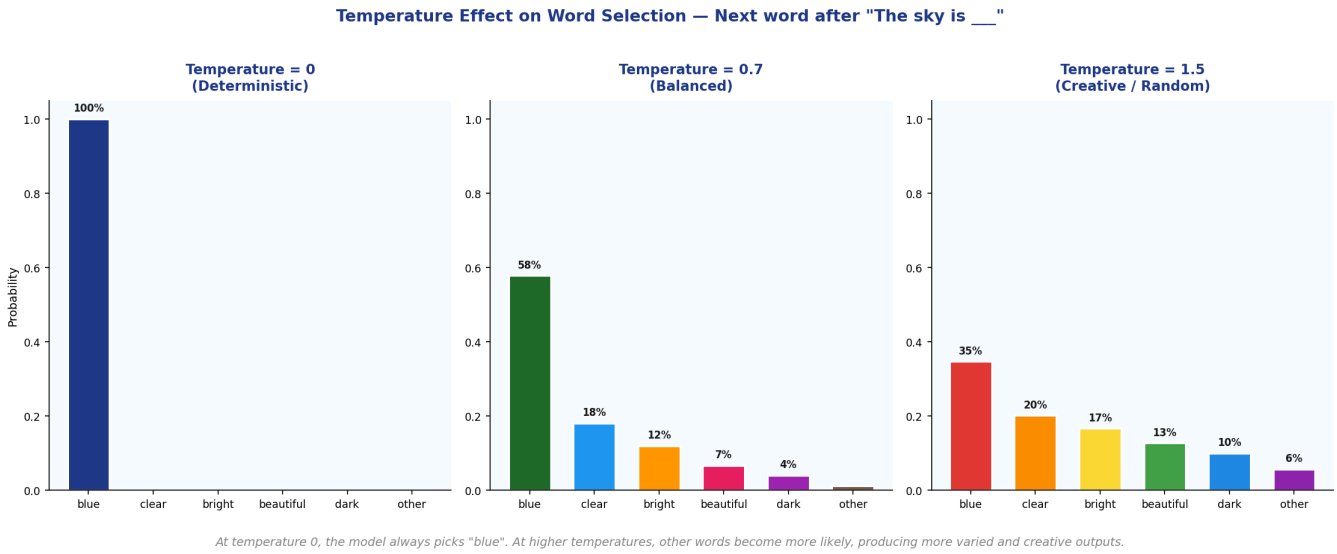

Figure 4.1 — At temperature 0, the model always picks 'blue'. As the temperature rises, other words become proportionally more likely, producing more varied outputs.

Temperature Explained

Temperature is a number, typically between 0 and 2, that controls how adventurous the model is when choosing the next word. A low temperature makes the model more likely to choose the highest-probability word — more predictable and conservative. A high temperature spreads the probability more evenly across words — more creative, surprising, and potentially unpredictable.

Temperature Setting	Behaviour and Best Use Cases
0.0 — Zero	Always picks the highest-probability word. Fully deterministic. Best for factual Q&A and code generation.
0.3 — Low	Mostly predictable with slight variation. Good for summarisation where accuracy is paramount.

0.7 — Medium	Balanced creativity and coherence. Suitable for general-purpose conversational AI.
1.0 — High	More creative and varied responses. Good for brainstorming and creative writing.
1.5+ — Very High	Highly unpredictable, potentially incoherent. Occasional experimental creative use.

For testers, temperature has profound implications. A system running at temperature 0 will produce the same output every time for the same input, making traditional automated testing feasible. A system running at higher temperatures will produce variable outputs, requiring a fundamentally different approach to quality assessment — one that evaluates quality rather than exact match.

4.3 Implications for Users

Understanding probability and temperature helps explain several behaviours you may observe when using AI systems:

- If you ask the same question twice and get different answers, the model is sampling from its probability distribution. Both answers may be equally valid.
- If you set the temperature to 0 and ask the same question twice, you should get the same answer — because there is no randomness.
- Very creative or unusual outputs are the result of the model exploring lower-probability word choices, which happens more frequently at higher temperatures.
- The model can generate text that sounds confident and fluent but is factually incorrect. Fluency and accuracy are separate dimensions.

Part 5 — Real-World Applications of AI

— Where AI Is Being Used Today —

Large Language Models and AI systems are no longer experimental technologies confined to research laboratories. They are actively deployed in products and services that millions of people use every day. In this section, we explore the most significant application areas, with a focus on understanding how they work and what testing challenges each one presents.

5.1 Conversational AI and Chatbots

The most visible application of LLMs is conversational AI — systems that can hold a flowing, contextual conversation with a user in natural language. ChatGPT, Claude, Gemini, and many others fall into this category. These systems can be used for an enormous variety of tasks: answering questions, helping with homework, drafting emails, explaining complex topics, and much more.

What makes modern conversational AI different from older chatbots is its ability to maintain context across a multi-turn conversation. The system remembers what was said earlier in the conversation and uses that context to inform its responses. Ask a chatbot to 'make it shorter', and it knows you are referring to the text it just generated, not some arbitrary text.

How Conversational Context Works

Every time you send a message in a conversation, the AI receives not just your new message, but the entire conversation history as input. This is why the context window matters so much — it determines how far back in the conversation the AI can 'see'. A very long conversation may eventually exceed the context window, causing the AI to appear to forget earlier exchanges.

5.2 AI Coding Assistants

One of the most transformative applications of LLMs for professionals is AI-powered coding assistance. Tools like GitHub Copilot, Amazon CodeWhisperer, and Cursor use LLMs trained specifically on code to help developers write, complete, explain, and debug software.

A developer might type a comment describing a function, and the AI generates the full implementation. Or they might paste a block of code with a bug, and the AI identifies the error and suggests a fix. These tools have dramatically increased developer productivity in many organisations.

Testing AI coding assistants involves checking not just whether the generated code runs, but whether it is correct, secure, efficient, and follows good software engineering practices. A function might pass basic tests but contain subtle security vulnerabilities or perform poorly with large inputs.

5.3 Retrieval-Augmented Generation (RAG)

One of the most important architectural patterns for enterprise AI applications is Retrieval-Augmented Generation, commonly known as RAG. This approach addresses one of the fundamental limitations of standard LLMs: their knowledge is frozen at the time of training.

A standard LLM cannot tell you about events that occurred after its training cutoff date. It cannot answer questions about your company's internal documents, policies, or data — unless that information was in its training data. RAG solves this problem by connecting the LLM to an external knowledge source.

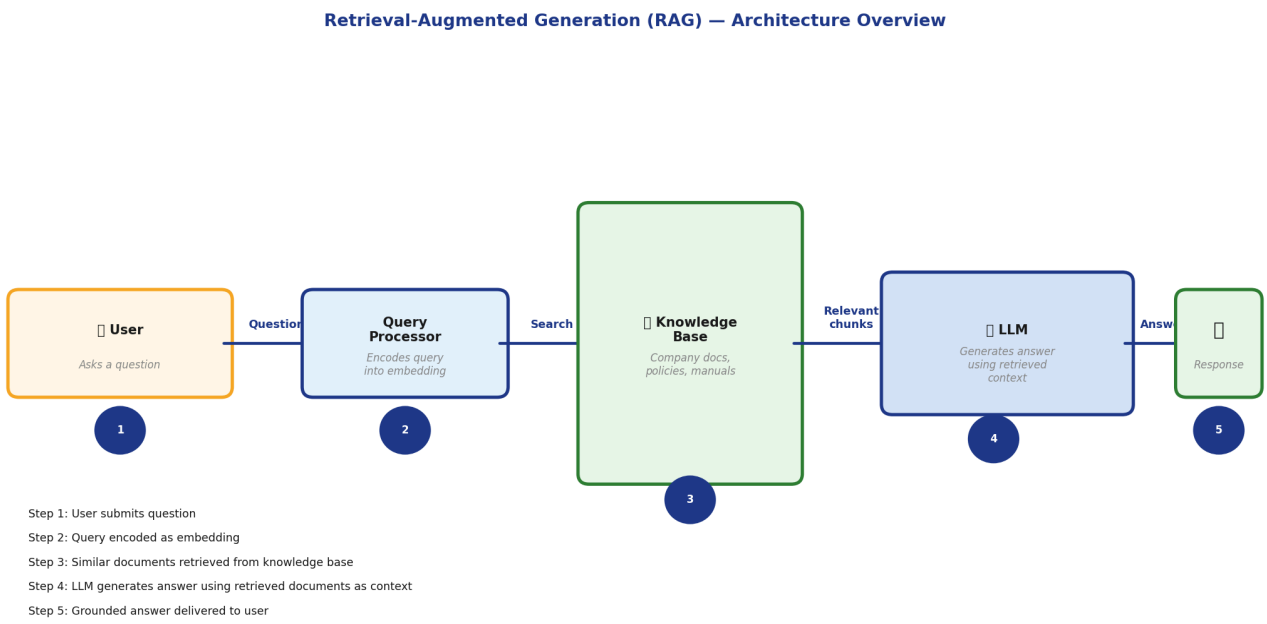

Figure 5.1 — The RAG architecture: the user's question triggers a document search, and the LLM generates an answer grounded in retrieved content

How RAG Works — Step by Step

Step 1: The user submits a question.Step 2: The system searches a database of documents (e.g., company policies, product manuals, research papers) for relevant content.Step 3: The most relevant documents or excerpts are retrieved.Step 4: The retrieved content is provided to the LLM as additional context, alongside the original question.Step 5: The LLM generates an answer grounded in the retrieved information, rather than relying solely on its training data.

RAG is used in enterprise knowledge bases, customer service systems, document question-answering tools, medical information systems, and legal research platforms. Testing RAG systems involves additional complexity: the tester must verify not only the quality of the generated response, but also the quality of the retrieval step — did the system retrieve the right documents?

5.4 Summary of Key Application Areas

Application Area	Description and Testing Focus
Conversational AI	Multi-turn dialogue systems. Test context retention, factual accuracy, and appropriate refusals.
Coding Assistants	Code generation and explanation. Test correctness, security, efficiency, and edge cases.
RAG Systems	Knowledge-grounded Q&A. Test retrieval quality, groundedness, and handling of missing information.
Content Generation	Writing, summarisation, translation. Test for quality, tone, plagiarism, and consistency.
Classification & Analysis	Sentiment analysis, intent detection. Test for accuracy, bias, and edge cases.

Part 6 — Why Testing AI is Fundamentally Different

— The New Frontier of Quality Assurance —

This is the section that most directly connects everything we have learned so far to the practice of software quality and testing. Testing AI systems requires us to completely rethink assumptions that have been foundational to software testing for decades. Let us explore why.

6.1 Traditional Software Testing — A Quick Review

In traditional software development, testing is based on a deceptively simple principle: for a given input, you know what output the system should produce. You write test cases that specify the expected output, run them against the system, and check whether the actual output matches the expected output.

This approach works beautifully for deterministic systems. Consider a login screen. If the user enters the correct username and password, the system should grant access. If the password is wrong, the system should deny access and display an error message. These rules never change. A test written today will be valid tomorrow and in five years.

The Foundation of Traditional Testing

Traditional software testing relies on the existence of a known, correct expected output for every test input. This is called the 'test oracle' — the authoritative source of what the correct answer is. In traditional software, the oracle is the specification: a document written by humans that defines exactly how the software should behave.

6.2 The Oracle Problem in AI Testing

Now apply this logic to an AI system. A user asks: 'Explain the water cycle in simple terms.' What is the correct response? There are hundreds, perhaps thousands, of responses that would be equally correct. They might differ in vocabulary, structure, level of detail, examples used, and tone. None of them is 'the right answer' in the way that '4' is the right answer to '2 + 2'.

This is known as the Oracle Problem, and it is the central challenge of AI testing. We often cannot define a single correct expected output. Therefore, we cannot use the fundamental test mechanism — compare actual output to expected output — in its traditional form.

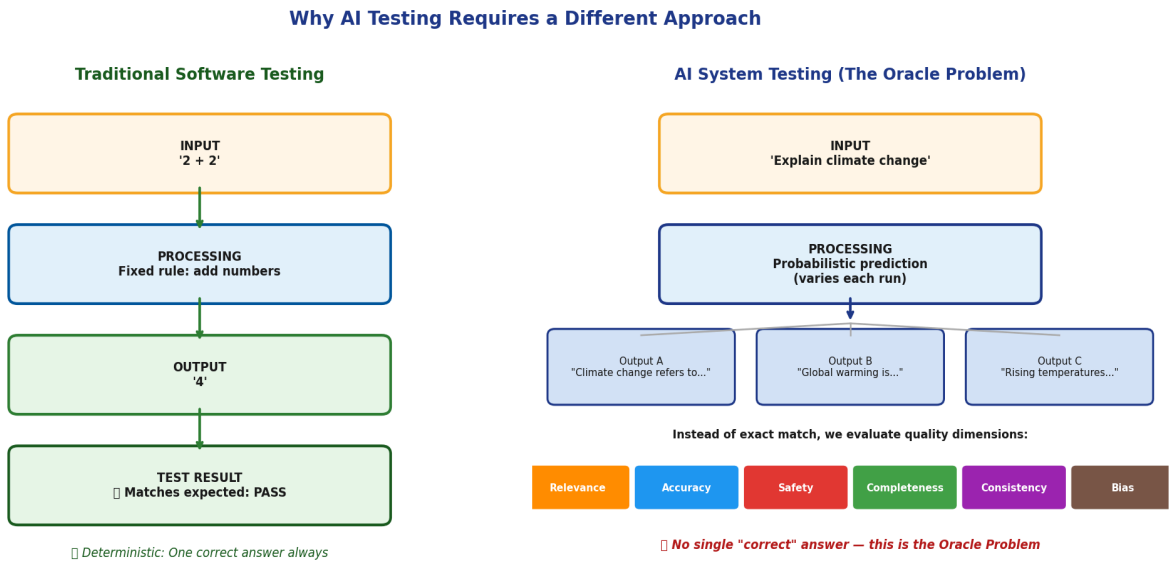

Figure 6.1 — Traditional testing compares to a single expected output. AI testing must evaluate quality across multiple valid responses.

Testing Dimension	What We Are Evaluating
Relevance	Does the response actually address what was asked?
Accuracy	Is the factual content of the response correct?
Completeness	Has the response covered the important aspects of the topic?
Safety	Does the response contain harmful, offensive, or dangerous content?
Bias	Does the response treat different groups and perspectives fairly?
Groundedness (RAG)	Is the response grounded in the provided source documents?
Consistency	Does the system give consistent responses to equivalent questions?
Appropriate refusals	Does the system correctly refuse inappropriate requests?

6.3 Prompt Sensitivity — Small Changes, Big Differences

Another important characteristic of LLM behaviour that testers must understand is prompt sensitivity. The way you phrase a question can significantly affect the answer you receive — sometimes in ways that seem disproportionate to the size of the change.

Example — Prompt Sensitivity

All three of the following prompts ask for the same information, but they may produce noticeably different responses: Prompt A: "Explain the water cycle."Prompt B: "Explain the water cycle in two sentences."Prompt C: "Explain the water cycle to a 10-year-old."Prompt A might produce a comprehensive multi-paragraph explanation. Prompt B will produce a brief summary. Prompt C will use simpler vocabulary and analogies. The underlying question is identical, but the phrasing shapes the output profoundly.

This has major implications for testers. It means that a test suite must consider variations in phrasing, not just variations in content. Common categories of prompt variation that testers should consider include:

- Instruction specificity — how precisely the task is described
- Role assignment — asking the AI to respond as an expert, a teacher, a novice, etc.
- Format specification — asking for bullet points, paragraphs, tables, code, etc.
- Length constraints — asking for brief or detailed responses
- Tone requests — asking for formal, casual, empathetic, or technical language

6.4 Non-Determinism and Regression Testing

In traditional software, regression testing is straightforward. You keep a set of test cases from previous releases. Each time a new version is released, you run those test cases and verify that the outputs have not changed.

In AI testing, this approach breaks down. Because the system's outputs are non-deterministic at temperatures above zero, a response today might be slightly different from a response tomorrow, even if nothing has changed in the system. Is that a regression? Usually not. But if the new response is consistently less accurate, less helpful, or less safe — then yes, something may have changed for the worse.

How to Handle Non-Determinism in Testing

Rather than testing for exact output match, AI regression testing typically involves:1. Running the same prompt multiple times and evaluating the distribution of outputs.2. Using automated scoring metrics (relevance, accuracy, safety scores) to evaluate output quality.3. Using AI judges — another LLM specifically configured to evaluate the quality of outputs against defined criteria.4. Comparing aggregate scores across model versions, rather than individual outputs.5. Maintaining a 'golden dataset' of inputs with defined quality thresholds, rather than exact expected outputs.

6.5 Key Challenges Unique to AI Testing

Core Challenges in AI Testing
1. The Oracle Problem: There is no single correct expected output for most AI queries. Testing must evaluate quality dimensions rather than an exact match.
2. Non-Determinism: The same input can produce different outputs across runs, making traditional automated testing difficult.
3. Prompt Sensitivity: Small changes in phrasing can produce large changes in the output, requiring comprehensive prompt variation testing.
4. Emergent Behaviours: AI systems can exhibit unexpected behaviours that were not explicitly programmed.
5. Bias and Fairness: Models can reflect the biases present in their training data, requiring specific fairness and equity testing.
6. Hallucination: Models can generate confident-sounding but factually incorrect content, requiring accuracy verification at scale.
7. Adversarial Inputs: Carefully crafted inputs can manipulate model behaviour in unexpected ways (prompt injection).
8. Context Window Limits: Model behaviour can change significantly with very long inputs or conversations.

Part 7 — Key Terms and Definitions

— Your AI Vocabulary Reference —

This section provides a comprehensive glossary of the key terms introduced in this module. Keep this section bookmarked — you will refer to it frequently as you progress through the course.

Term	Definition
Artificial Intelligence (AI)	Computer systems are designed to perform tasks requiring human-like intelligence, such as language understanding, pattern recognition, and decision-making.
Machine Learning (ML)	A form of AI where systems learn patterns from data, rather than following explicitly programmed rules.
Deep Learning	A subset of machine learning using multi-layered neural networks capable of learning complex patterns from large datasets.
Neural Network	A computational architecture loosely inspired by the human brain, consisting of interconnected layers of nodes that process information.
Large Language Model (LLM)	A neural network trained on vast amounts of text data to understand and generate human language at scale.
Token	The basic unit of text that a language model processes. May be a word, part of a word, or a punctuation mark.
Tokenization	The process of breaking text into tokens before a language model can process it.
Context Window	The maximum amount of text (measured in tokens) that a language model can process in a single interaction.
Embedding	A numerical representation (vector) of a token that encodes its meaning and relationships to other tokens.
Attention Mechanism	The component of a modern language model that allows it to relate any token to any other token in the input, capturing context and dependencies.
Transformer	The neural network architecture that uses the attention mechanism, underlying most modern LLMs.
Temperature	A parameter that controls the randomness of a language model's output. Higher temperature produces more creative and varied outputs.
Probability Distribution	The set of probabilities the model assigns to each possible next token when generating text.
Hallucination	When an AI system generates content that is factually incorrect but presented with apparent confidence.

Prompt	The input text is provided to an AI system to guide its response.
Prompt Engineering	The practice of carefully designing prompts to achieve desired outputs from an AI system.
Prompt Sensitivity	The phenomenon is where small changes in prompt wording can produce significantly different outputs.
RAG (Retrieval-Augmented Generation)	An architecture where an LLM is connected to an external knowledge base, retrieving relevant documents before generating a response.
Deterministic System	A system that always produces the same output for the same input.
Non-Deterministic System	A system that may produce different outputs for the same input across different runs.
Oracle Problem	The challenge in AI testing is defining what the correct expected output is when many different outputs may be equally valid.
Bias	Systematic patterns in AI outputs that unfairly favour or disadvantage particular groups, often reflecting biases in training data.
Ground Truth	The verified correct answer or label is used to evaluate the accuracy of an AI system's outputs.

Part 8 — Module Summary and Review

— What You Have Learned —

You have now completed Module 1. This has been a significant amount of material, and it is worth taking time to consolidate what you have learned. The following summary organises the key ideas from each section of the module.

8.1 Section-by-Section Summary

Part 1 — What Is Artificial Intelligence?

AI systems differ from traditional software in that they learn from data rather than following explicit rules. They can perform tasks requiring human-like intelligence, but they do not truly 'understand' the world. They are sophisticated pattern-recognition and prediction machines.

Part 2 — The Evolution of AI

AI evolved through four phases: rule-based systems (explicit instructions), machine learning (learning from labelled data), deep learning (neural networks with multiple layers), and large language models (massive scale text prediction). Each phase brought greater capability and new testing challenges.

Part 3 — How LLMs Work

Language models work through three key mechanisms: tokenisation (breaking text into processable units), embeddings (converting tokens into meaningful numerical representations), and attention (relating any word to any other word to capture context). Together, these enable rich language understanding.

Part 4 — Probability and Temperature

LLMs generate text by predicting the probability of each possible next token and sampling from that distribution. Temperature controls how adventurously the model samples. Low temperature produces consistent, predictable outputs; high temperature produces creative, varied outputs.

Part 5 — Real-World Applications

Key applications include conversational AI (chatbots), coding assistants, and RAG systems. Each has distinct characteristics and testing requirements. RAG addresses the knowledge cutoff limitation by retrieving external documents before generation.

Part 6 — Why AI Testing Is Different

AI testing faces the Oracle Problem (no single correct answer), non-determinism (variable outputs), prompt sensitivity (phrasing matters enormously), and emergent behaviours. Traditional exact-match testing must be replaced by quality-dimension evaluation.

8.2 Diagram Reference Guide

Use this guide to quickly find the key diagrams in this module:

Figure	What It Shows
Figure 1.1 — AI Evolution Timeline	The four phases from rule-based systems to LLMs, with historical context
Figure 2.1 — Neural Network Architecture	How layers of a neural network process information from input to output
Figure 3.1 — Tokenisation	How text is split into tokens before a model can process it
Figure 3.2 — Word Embeddings	How similar words cluster together in the mathematical embedding space
Figure 3.3 — Attention Mechanism	How the model resolves pronoun references by attending to relevant words
Figure 4.1 — Temperature Effect	How temperature setting changes the probability distribution of word selection
Figure 5.1 — RAG Architecture	The five-step process of retrieval-augmented generation
Figure 6.1 — Testing Comparison	Why AI testing requires quality-dimension evaluation rather than exact-match testing

8.3 The Single Most Important Insight

Remember This Above All Else

An AI language model does not know things. It predicts things. It does not retrieve stored answers from a database. It generates text token by token, based on statistical patterns learned from its training data. This is why it can be creative, why it can make mistakes, why it can produce different answers to the same question, and why testing it requires entirely different approaches from testing traditional software.As AI professionals — whether as testers, developers, or users — our fundamental job is to evaluate the quality and reliability of these probabilistic predictions at scale.

Part 9 — Self-Assessment Questions

— Test Your Understanding —

The following questions are designed to help you assess your understanding of the material covered in this module. Answer each question in your own words before checking the answer. The goal is not to memorise definitions, but to ensure you can explain concepts clearly and in your own language.

9.1 Conceptual Understanding

Questions — Answer Before Reading Further
Q1. What is the fundamental difference between traditional software and an AI system? Give one real-world example of each.
Q2. Explain the four phases in the evolution of AI. What was the key innovation in each phase?
Q3. In your own words, explain what a token is and why tokenisation is necessary.
Q4. What is an embedding, and what is the significance of similar words having similar embeddings?
Q5. Explain the attention mechanism. Use the pronoun example (or create your own) to illustrate why it matters.

Q6. Why does a language model give different answers to the same question? Name the two factors that determine this.
Q7. What is the Oracle Problem in AI testing? Why does it make traditional testing approaches insufficient?
Q8. What is prompt sensitivity? Why does it matter for quality assurance professionals?

9.2 Application Questions

Think Through These Scenarios
Scenario 1: You are testing a customer service chatbot for an airline. The chatbot should answer questions about flight delays, baggage policies, and booking changes. What testing dimensions would you use to evaluate its quality? How would you handle the Oracle Problem?
Scenario 2: Your team has built a RAG system for a law firm that answers questions about legal precedents. A tester finds that the system sometimes gives answers not supported by the retrieved documents. What is this problem called, and how would you approach testing for it systematically?
Scenario 3: You run the same test prompt 10 times at a temperature of 0.7 and get 10 slightly different responses. Is this a defect? How would you decide whether the system is behaving correctly?
Scenario 4: A junior tester argues that since AI outputs are non-deterministic, it is impossible to do meaningful automated testing. How would you respond to this argument?

9.3 Sample Answers

Here are concise sample answers to the conceptual questions. Your answers may be worded differently — what matters is that the core ideas are present.

Sample Answers
A1. Traditional software follows fixed, pre-programmed rules and always produces the same output for the same input (e.g., a login system). AI systems learn from data and can produce flexible, context-aware outputs that may vary (e.g., a chatbot that generates unique responses each time).
A2. Phase 1 (Rule-based): Programmers write all rules explicitly. Phase 2 (Machine Learning): Systems learn patterns from labelled data. Phase 3 (Deep Learning): Multi-layer neural networks detect complex patterns. Phase 4 (LLMs): Massive models trained on text data to predict language at scale.

A3. A token is a small unit of text — a word, part of a word, or punctuation — that a language model processes. Tokenisation is necessary because models cannot process raw text directly; they need a standardised, mathematical representation.
A4. An embedding is a numerical vector representing a token's meaning. The significance is that semantically similar words have mathematically similar embeddings, allowing the model to understand relationships between concepts without explicit programming.
A5. The attention mechanism allows the model to calculate the relationship between every pair of words simultaneously. In 'The trophy does not fit in the suitcase because it is too small', the model determines that 'it' refers to 'suitcase' by strongly attending to the relationship between those words, given the semantic context.
A6. Different answers result from (1) probabilistic sampling — the model chooses from a distribution of possible next tokens rather than always picking the top-ranked one, and (2) temperature, which controls how broadly the model samples from that distribution.
A7. The Oracle Problem is the challenge that, for most AI queries, there is no single correct expected output. Multiple valid responses exist. This makes traditional 'compare actual to expected' testing impossible — we must instead evaluate quality dimensions such as relevance, accuracy, and safety.
A8. Prompt sensitivity means that small changes in how a question is phrased can produce significantly different outputs. For QA professionals, this means testing must include systematic variation of prompts, not just variation of content, to ensure the system behaves reliably across the range of inputs real users will submit.

Part 10 — Further Reading and Module Close

— Extending Your Knowledge —

10.1 Recommended Reading

To deepen your understanding of the concepts covered in this module, the following resources are recommended. They are ordered from most accessible to most technical.

Resource	Why It Is Recommended
"Attention Is All You Need" (Vaswani et al., 2017)	The landmark paper that introduced the transformer architecture. The abstract and introduction are accessible without a deep mathematical background.
"The Illustrated Transformer" by Jay Alammar (blog post)	An outstanding visual explanation of how transformers and attention work. Freely available online.
Anthropic's Model Cards and Research Blog	Provides insight into how real LLMs are built, evaluated for safety, and deployed responsibly.

"AI Snake Oil" by Arvind Narayanan & Sayash Kapoor	A clear-headed examination of where AI works well and where it is overhyped. Essential for critical thinking.
OpenAI's Playground and API documentation	Hands-on experience with temperature settings and prompt variation. Experiment directly with the API.

10.2 Hands-On Practice

Reading and understanding concepts is only one part of learning. The following practical exercises will cement your understanding:

- Experiment with temperature: Use an AI system that allows temperature adjustment (such as the OpenAI Playground or Anthropic Console). Ask the same question at temperatures 0, 0.5, and 1.0. Observe and document the differences.
- Explore prompt sensitivity: Take a simple question and rephrase it five different ways. Record the outputs. Identify which changes in phrasing produced the most significant changes in output.
- Test the context window: Start a very long conversation with a chatbot. After many exchanges, refer back to something said very early. Does the system remember it? What happens as the conversation grows longer?
- Probe for hallucination: Ask a chatbot about a niche historical fact you can verify independently. Ask the same question in multiple ways. Document any inconsistencies or inaccuracies.
- Design test cases: Choose a specific AI application. Design 10 test cases that cover different quality dimensions. For each test, define how you would evaluate whether the output is acceptable.

10.3 A Closing Thought

The Mindset of an AI Quality Professional

Traditional software testing asked: 'Is the output correct?'
AI quality assurance asks: 'Is the output good enough and good enough for whom?'
This shift from binary correctness to multidimensional quality is not just a technical change. It is a philosophical one. It requires us to think carefully about what we value, what we consider acceptable, and who might be harmed when AI systems fail. As you progress through this course, carry this mindset with you. You are not just learning to find bugs. You are learning to ask better questions about one of the most powerful and consequential technologies ever built.

End of Module 1

Understanding AI and Large Language Models